

AEDS II - Lista Completa de Preparação para a Reavaliação

Diagnóstico baseado nos materiais enviados

Foram analisados os quatro conjuntos enviados: **Slides AEDS 2, Códigos, Exercícios/Laboratórios e Provas de Reavaliação**. A lista abaixo foi construída priorizando, nesta ordem, as provas antigas, o padrão dos códigos do professor, os laboratórios e os slides.

Conteúdos efetivamente encontrados

- Fundamentos de análise: contagem de operações, somatórios, fórmula fechada, prova por indução, melhor/pior caso e notação assintótica.
- Pesquisa sequencial e binária.
- Estruturas lineares sequenciais: lista, lista ordenada, pilha e fila circular.
- Ordenação interna: seleção, Bubble Sort, Insertion Sort, Shellsort, Quicksort, Merge Sort, Heapsort, Counting Sort e Radix Sort; também aparecem ordenação parcial e discussão de paralelismo.
- Estruturas flexíveis: pilha, fila, lista simples, lista dupla e matriz encadeada.
- Árvore binária de busca: inserção, pesquisa, caminhamentos, remoção, altura, maior/menor, soma, contagens e comparação de árvores.
- Estruturas híbridas: árvore de árvores, árvore de listas, matriz de listas e combinações de várias estruturas.
- Balanceamento: AVL, árvore 2-3-4 e árvore alvinegra/bicolor.
- Tabelas hash: área de reserva, rehash, encadeamento e estrutura “doidona” com múltiplos destinos.
- TRIE e PATRICIA, incluindo versões em que os filhos são gerenciados por hash, lista encadeada ou árvore binária.

Padrão de implementação do professor

- A linguagem predominante nas provas e códigos centrais é **Java**.
- É frequente um método público que chama um auxiliar recursivo privado.
- Nomes recorrentes: No, Celula, raiz, elemento, esq, dir, primeiro, ultimo, array, n e resp.
- Operações inválidas ou duplicadas normalmente lançam Exception.
- A AVL do material usa nível: nó nulo tem nível 0 e um nó existente tem $1 + \max(\text{nível}(\text{esq}), \text{nível}(\text{dir}))$. O fator é $\text{nível}(\text{dir}) - \text{nível}(\text{esq})$.
- Na TRIE, o nó possui char elemento, No[] prox e boolean folha.
- Nas questões de prova, as classes e estruturas são fornecidas e o aluno implementa apenas um método ou uma pequena família de métodos.

Padrão observado nas reavaliações

As provas antigas combinam:

1. contagem exata de operações, somatórios e indução;
2. desenho passo a passo de AVL, 2-3-4 e alvinegra;
3. questões de provar ou refutar com justificativa;
4. adaptação de algoritmos de ordenação;
5. análise de melhor e pior caso de trechos de código;
6. implementação de métodos sobre estruturas prontas;
7. estruturas híbridas ou “doidonas” descritas por classes e diagramas;
8. manipulação de matriz encadeada e listas.

Prioridade recomendada de estudo

1. Métodos recursivos sobre árvores e estruturas híbridas.
 2. Inserção, remoção, pesquisa e desenho de ABB, AVL, 2-3-4 e alvinegra.
 3. Hash com reserva, rehash, encadeamento e estruturas doidonas.
 4. Alteração de Bubble Sort, Insertion Sort e Merge Sort, além de análise de complexidade.
 5. Listas, pilhas, filas e matriz encadeada.
 6. TRIE/PATRICIA e variantes de gerenciamento de filhos.
 7. Somatórios, fórmula fechada, indução e melhor/pior caso.
-

Módulo 1 - Questões conceituais e de análise

Questão 1 - Somatório e fórmula fechada

Nível: Básico

Considere dois laços: o externo executa para $i = 1..n$ e o interno para $j = 1..i$. Escreva o somatório do número de comparações do corpo, encontre a fórmula fechada e informe a ordem assintótica. Não basta responder apenas $O(n^2)$.

Questão 2 - Função exata e ordem assintótica

Nível: Intermediário

Um método executa $4n^2 + 7n + 12$ atribuições. Explique a diferença entre função de complexidade exata e classe assintótica. Determine O , Ω e Θ e justifique por que constantes e termos de menor grau são descartados na análise assintótica.

Questão 3 - Prove ou refute - bloco I

Nível: Prova de reavaliação

Prove ou refute, sempre com justificativa: (a) uma pesquisa em lista encadeada nunca pode ser constante; (b) toda ABB com mais de um elemento é AVL; (c) a inserção em árvore balanceada nunca pode ser logarítmica; (d) busca binária em vetor desordenado pode ter pior caso $O(\log n)$ sem pré-processamento; (e) a pesquisa em hash com encadeamento é sempre $O(1)$ no pior caso.

Questão 4 - Pesquisa sequencial e binária

Nível: Básico

Compare pesquisa sequencial e binária quanto a pré-condições, melhor caso, pior caso e custo de manutenção do vetor. Explique por que um vetor ordenado não torna automaticamente toda operação de inserção $O(\log n)$.

Questão 5 - Lista sequencial

Nível: Intermediário

Para uma lista sequencial com n elementos, indique o custo de inserir e remover no início, no fim e em uma posição arbitrária. Separe o custo de localizar a posição do custo de movimentar os elementos.

Questão 6 - Fila circular

Nível: Intermediário

Explique o papel dos índices primeiro e ultimo na fila circular do material. Justifique por que o vetor possui uma posição extra e como são detectados os estados de fila vazia e fila cheia.

Questão 7 - Pilha, fila e lista

Nível: Básico

Para cada cenário, escolha a estrutura mais adequada e justifique: desfazer ações; atendimento por ordem de chegada; verificar parênteses balanceados; inserir e remover em posições arbitrárias; percurso em largura de árvore.

Questão 8 - Estruturas sequenciais e flexíveis

Nível: Intermediário

Compare versões sequenciais e flexíveis de lista, pilha e fila em relação a memória, acesso, movimentações, alocação, localidade de cache e risco de overflow.

Questão 9 - Lista simples e lista dupla

Nível: Intermediário

Explique quais referências precisam ser alteradas ao remover uma célula do meio de uma lista simplesmente encadeada e de uma lista duplamente encadeada. Compare os custos quando se possui apenas um ponteiro para a célula a remover.

Questão 10 - Matriz encadeada

Nível: Avançado

A matriz flexível do material conecta cada célula por sup, inf, esq e dir. Explique como caminhar pela diagonal principal e por que as células-cabeça, quando existirem, devem ser tratadas separadamente dos dados.

Questão 11 - Estabilidade e uso de memória

Nível: Intermediário

Classifique Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, QuickSort, HeapSort, Counting Sort e Radix Sort quanto a estabilidade e uso de memória auxiliar. Quando houver dependência da implementação, deixe isso explícito.

Questão 12 - Bubble, Insertion e Merge

Nível: Prova de reavaliação

Compare melhor e pior caso de Bubble Sort otimizado, Insertion Sort e Merge Sort. Para cada algoritmo, descreva um arranjo que produza o melhor caso e outro que produza o pior caso em comparações e movimentações.

Questão 13 - Selection, Quick, Heap, Counting e Radix

Nível: Avançado

Explique por que Selection Sort continua quadrático em comparações mesmo em vetor ordenado; quando QuickSort degrada para quadrático; por que HeapSort mantém $O(n \log n)$; e quais hipóteses permitem Counting Sort e Radix Sort terem desempenho próximo do linear.

Questão 14 - Escolha do algoritmo de ordenação

Nível: Intermediário

Escolha um algoritmo apropriado para cada cenário e justifique: vetor quase ordenado e pequeno; grande volume com necessidade de estabilidade; chaves inteiras em intervalo pequeno; memória auxiliar muito limitada; dados possivelmente já ordenados e pivô sempre no início.

Questão 15 - Propriedades da ABB

Nível: Básico

Explique a propriedade global de uma ABB e por que conferir apenas os filhos imediatos não é suficiente para validar a árvore. Compare as complexidades de pesquisa, inserção e remoção em uma árvore balanceada e em uma degenerada.

Questão 16 - Remoção em ABB

Nível: Intermediário

Descreva os três casos de remoção em ABB. Explique a função de maiorEsq utilizada no código do professor e por que ela preserva a ordenação.

Questão 17 - Padrões de recursão em árvore

Nível: Avançado

Compare três padrões: percorrer todos os nós e somar resultados; percorrer apenas um ramo usando ordenação; retornar informação de baixo para cima. Dê um exemplo de método de cada padrão.

Questão 18 - AVL: nível, fator e rotações

Nível: Intermediário

Use a convenção do material: nível nulo 0 e fator nível(dir)-nível(esq). Explique os quatro casos de rotação, quais fatores levam a cada caso e quais níveis precisam ser atualizados depois da rotação.

Questão 19 - Árvore 2-3-4 e alvinegra

Nível: Avançado

Explique como 2-nós, 3-nós e 4-nós podem ser representados em uma árvore alvinegra. Compare fragmentação na descida e fragmentação por ascensão na árvore 2-3-4.

Questão 20 - Invariantes da alvinegra

Nível: Avançado

Apresente as propriedades de cor necessárias para manter a altura logarítmica. Explique o significado de um 4-nó na representação binária e o papel de recoloração e rotação.

Questão 21 - Estratégias de colisão em hash

Nível: Intermediário

Compare área de reserva, rehash e encadeamento separado em inserção, pesquisa, remoção, aproveitamento de memória e comportamento no pior caso. Explique o papel do fator de carga.

Questão 22 - TRIE e PATRICIA

Nível: Intermediário

Compare pesquisa exata, pesquisa por prefixo e listagem por prefixo em hash, TRIE e PATRICIA. Explique a função do campo folha e por que alcançar o último caractere não é suficiente para confirmar uma palavra.

Questão 23 - Complexidade de estruturas híbridas

Nível: Avançado

Uma pesquisa passa por uma ABB de categorias, uma tabela hash e uma lista de colisões. Escreva a complexidade em termos da altura h , do custo da hash e do comprimento k da lista. Explique como expressar melhor e pior caso sem esconder as camadas.

Questão 24 - Prove ou refute - bloco II

Nível: Desafio

Prove ou refute: (a) toda AVL é ABB; (b) toda ABB é alvinegra; (c) Merge Sort é quadrático no pior caso; (d) Radix Sort é sempre $O(n)$ independentemente do tamanho das chaves; (e) uma pilha é suficiente para validar parênteses balanceados; (f) uma TRIE pode reconhecer prefixo sem visitar todas as palavras; (g) toda pesquisa em hash é constante; (h) uma lista dupla permite remoção constante se já houver referência direta para a célula.

Módulo 2 - Desenho, simulação e rastreamento

Questão 25 - Lista sequencial: deslocamentos

Nível: Básico

Uma lista de capacidade 10 contém [4, 7, 9, 15] com $n=4$. Mostre o vetor e n após: `inserirInicio(2)`, `inserir(8,3)`, `remover(4)` e `removerFim()`. Indique todas as movimentações.

Questão 26 - Fila circular: retorno ao início

Nível: Intermediário

Considere fila circular de capacidade lógica 5 (vetor de tamanho 6), inicialmente vazia. Simule: `inserir 10,20,30,40`; `remover` duas vezes; `inserir 50,60,70`; `remover`; `inserir 80`. Mostre primeiro, ultimo e o vetor após cada operação.

Questão 27 - Pilha e fila em sequência

Nível: Básico

Uma pilha e uma fila começam vazias. Execute em ambas: `inserir 3`, `inserir 8`, `inserir 5`, `remover`, `inserir 2`, `remover`. Desenhe o estado após cada etapa e informe os valores removidos.

Questão 28 - Lista simples: inserção e remoção

Nível: Intermediário

Desenhe uma lista simples com célula-cabeça após `inserir 5`, `9` e `12` no fim; `inserir 7` na posição 1; `remover` o início; `remover` a posição 1. Identifique primeiro, ultimo e cada alteração de prox.

Questão 29 - Lista dupla: religação

Nível: Intermediário

Desenhe uma lista dupla com cabeça contendo 4 <-> 8 <-> 15 <-> 16. Remova 8, insira 10 entre 15 e 16 e depois remova o fim. Mostre ant, prox, primeiro e ultimo.

Questão 30 - Matriz encadeada e diagonal

Nível: Prova de reavaliação

Dada uma matriz encadeada 3x3 com valores 1 a 9 em ordem de linhas, marque o caminho da diagonal principal e desenhe a lista duplamente encadeada resultante da concatenação das listas associadas às células 1, 5 e 9. Desconsidere células-cabeça.

Questão 31 - Bubble Sort passo a passo

Nível: Intermediário

Aplique Bubble Sort ao vetor [7, 2, 5, 1, 4]. Mostre o vetor após cada comparação que realiza troca, após cada passagem completa e a posição da última troca.

Questão 32 - Insertion Sort com regra ímpar/par

Nível: Prova de reavaliação

Simule uma versão de Insertion Sort que deixa os ímpares antes dos pares e mantém cada grupo em ordem crescente. Use [4,3,8,6,1,2,9,0,5,7] e mostre o vetor ao final de cada iteração externa.

Questão 33 - Merge Sort

Nível: Intermediário

Mostre todas as divisões e intercalações do Merge Sort para [38,27,43,3,9,82,10]. Em cada intercalação, indique os subvetores e a sequência de comparações.

Questão 34 - Particionamento do QuickSort

Nível: Avançado

Use o particionamento do código do professor (ponteiros i e j e pivô no meio) sobre [9,4,8,3,1,2,5]. Mostre movimentos de i, j, trocas e subintervalos recursivos.

Questão 35 - Construção e percursos da ABB

Nível: Básico

Insira em uma ABB: 15, 8, 22, 4, 12, 18, 30, 10, 14. Desenhe a árvore e escreva os caminhamentos central, pré-ordem e pós-ordem.

Questão 36 - Remoções na ABB

Nível: Intermediário

Na árvore da questão anterior, remova nesta ordem: 10 (folha), 12 (um filho após a primeira remoção), 15 (dois filhos). Desenhe cada estado e identifique a chamada de maiorEsq.

Questão 37 - TreeSort

Nível: Intermediário

Insira [7,3,9,1,5,8,10] em uma ABB e mostre como o caminhamento central produz o vetor ordenado. Conte inserções, visitas e posições preenchidas.

Questão 38 - Quatro rotações AVL

Nível: Intermediário

Desenhe a árvore após cada inserção e a rotação necessária para as sequências: (a) 30,20,10; (b) 10,20,30; (c) 30,10,20; (d) 10,30,20. Use a convenção de nível do material.

Questão 39 - AVL completa

Nível: Prova de reavaliação

Insira 4,20,8,12,2,18,14,10,6,22,16 em uma AVL. Após cada inserção, indique o primeiro nó desbalanceado, o tipo de rotação e o novo nível dos nós alterados. Desenhe a árvore final com fator de balanceamento.

Questão 40 - 2-3-4 com fragmentação na descida

Nível: Prova de reavaliação

Insira 4,20,8,12,2,18,14,10,6,22,16 em uma árvore 2-3-4, fragmentando 4-nós durante a descida. Mostre todos os estados em que ocorre fragmentação e a árvore final.

Questão 41 - 2-3-4 com fragmentação por ascensão

Nível: Avançado

Repita a sequência da questão anterior, mas permita o overflow na folha e faça a fragmentação por ascensão. Compare as etapas e desenhe a árvore final.

Questão 42 - Árvore alvinegra

Nível: Prova de reavaliação

Insira 4,20,8,12,2,18,14,10,6,22,16 usando o algoritmo alvinegro do material. Use B para branco e P para preto/colorido conforme definido na questão. Registre recolorações, rotações e árvore final.

Questão 43 - Hash: três estratégias

Nível: Intermediário

Para $m=7$, use $h(x)=x \bmod 7$ e valores 14,21,8,15,22,3,10. Desenhe: (a) hash com reserva de 3 posições; (b) rehash $reh(x)=(x+1) \bmod 7$; (c) encadeamento separado. Mostre pesquisas por 22 e 17.

Questão 44 - TRIE, PATRICIA e estrutura doidona

Nível: Desafio

Parte A: construa uma TRIE para BRASIL, BRASA, BRAVO, CHILE, CHINA, marcando finais.
Parte B: represente a versão PATRICIA comprimida. Parte C: desenhe uma estrutura em que uma árvore de letras aponta para uma hash por tamanho e cada colisão aponta para uma ABB de palavras; mostre o caminho de pesquisa por BRASA.

Módulo 3 - Código e implementação

Em todas as questões, mantenha o estilo do material: método público, auxiliar privado quando necessário, nomes de atributos fornecidos e análise de complexidade. Não apresente somente uma ideia; escreva o método solicitado.

Questão 45 - Contagem recursiva de maiúsculas

Nível: Básico

Implemente o método recursivo `public static int contarMaiusculas(String s)` sem usar laços. O método público deve chamar um auxiliar com índice. Informe a complexidade.

```
class Aquecimento {  
    public static int contarMaiusculas(String s) {  
        // implementar  
    }  
}
```

Questão 46 - Contagem exata de multiplicações

Nível: Prova de reavaliação

Para o método abaixo, escreva o somatório do número de multiplicações, encontre a fórmula fechada e prove-a por indução. Depois informe Θ .

```
public void calcula(int n) {  
    for (int i = 1; i <= n; i++) {  
        for (int j = 1; j <= i; j++) {
```

```
    int x = i * j;
    int y = x * 2;
}
}
```

Questão 47 - Inserção ordenada em lista sequencial

Nível: Básico

Na classe abaixo, implemente `public void inserirOrdenado(int x)` deslocando os elementos maiores. Lance exceção se a lista estiver cheia.

```
class Lista {
    private int[] array;
    private int n;

    public void inserirOrdenado(int x) throws Exception {
        // implementar
    }
}
```

Questão 48 - Remover todas as ocorrências

Nível: Intermediário

Implemente `public int removerTodos(int x)`, que remove todas as ocorrências de `x` de uma lista sequencial e retorna quantas foram removidas. Faça no máximo uma passagem principal e não use outro vetor.

```
class Lista {
    private int[] array;
    private int n;

    public int removerTodos(int x) {
        // implementar
    }
}
```

Questão 49 - Fila circular: soma recursiva

Nível: Intermediário

Usando a fila circular do material, implemente `public int somaRec()` sem modificar primeiro e ultimo. Crie um auxiliar recursivo que trate o retorno ao início do vetor.

```
class Fila {
    private int[] array;
```

```

private int primeiro, ultimo;

public int somaRec() {
    // implementar
}
}

```

Questão 50 - Parênteses balanceados

Nível: Intermediário

Implemente public static boolean balanceada(String s) usando uma pilha flexível de caracteres. Considere (), [] e {}. Ignore outros caracteres. Analise tempo e espaço.

Questão 51 - Pilha flexível: máximo recursivo

Nível: Intermediário

Na pilha flexível, implemente public int getMax() por recursão, sem remover elementos. Lance exceção se a pilha estiver vazia.

```

class Celula { int elemento; Celula prox; }
class Pilha {
    private Celula topo;
    public int getMax() throws Exception {
        // implementar
    }
}

```

Questão 52 - Inverter fila com pilha

Nível: Intermediário

Implemente public static void inverter(Fila fila) usando uma Pilha auxiliar e apenas as operações públicas inserir e remover. A fila deve terminar com a ordem invertida.

Questão 53 - Lista simples: inserção ordenada

Nível: Intermediário

Implemente public void inserirOrdenado(int x) em uma lista simples com célula-cabeça e ponteiro ultimo. Trate início, meio e fim sem criar uma segunda lista.

```

class Celula { int elemento; Celula prox; }
class Lista {
    private Celula primeiro, ultimo;
    public void inserirOrdenado(int x) {
        // implementar
    }
}

```

```
}  
}
```

Questão 54 - Lista simples: remover por valor

Nível: Intermediário

Implemente `public int removerValor(int x)` que remove a primeira ocorrência de `x` em uma lista simples com cabeça. Retorne o elemento removido e lance exceção se não existir. Atualize `ultimo` quando necessário.

Questão 55 - Lista dupla: palíndromo

Nível: Avançado

Implemente `public boolean isPalindromo()` comparando simultaneamente do início e do fim, sem copiar os elementos para vetor.

```
class CelulaDupla { int elemento; CelulaDupla ant, prox; }  
class ListaDupla {  
    private CelulaDupla primeiro, ultimo;  
    public boolean isPalindromo() {  
        // implementar  
    }  
}
```

Questão 56 - Matriz encadeada: diagonal unificada

Nível: Prova de reavaliação

Cada `CelulaMatriz` possui uma lista simples em início. Implemente `public CelulaDupla diagUnificada()` que cria uma única lista duplamente encadeada com todos os elementos das listas das células da diagonal principal, na ordem em que aparecem. Ignore células-cabeça das listas originais e não compartilhe células com a matriz.

```
class Matriz {  
    CelulaMatriz inicio;  
    int linha, coluna;  
    public CelulaDupla diagUnificada() {  
        // implementar  
    }  
}  
  
class CelulaMatriz { CelulaMatriz esq, dir, inf, sup; Celula inicio, fim; }  
class Celula { int elemento; Celula prox; }  
class CelulaDupla { int elemento; CelulaDupla prox, ant; }
```

Questão 57 - Bubble Sort com última troca

Nível: Prova de reavaliação

Reimplemente Bubble Sort para: (a) encerrar se uma passagem não fizer troca; (b) reduzir o próximo limite até a posição da última troca realizada. Conte comparações e movimentações.

Questão 58 - Insertion Sort: ímpares antes de pares

Nível: Prova de reavaliação

Altere o Insertion Sort para ordenar os ímpares antes dos pares e manter os dois grupos em ordem crescente. A comparação deve funcionar também para números negativos. Analise melhor e pior caso.

Questão 59 - Merge Sort decrescente

Nível: Intermediário

Adapte intercalar e mergesort para ordenar um vetor de inteiros em ordem decrescente, preservando estabilidade para elementos iguais. Mostre apenas os métodos alterados.

Questão 60 - Merge Sort de objetos

Nível: Avançado

Implemente a intercalação estável de objetos Aluno, ordenando por nota decrescente e, em caso de empate, por nome crescente.

```
class Aluno { String nome; double nota; }  
private void intercalar(int esq, int meio, int dir) {  
    // implementar  
}
```

Questão 61 - Corrigir Merge Sort

Nível: Intermediário

O método abaixo perde elementos. Identifique os erros e reescreva a versão correta.

```
private void intercalar(int esq, int meio, int dir) {  
    int[] tmp = new int[dir - esq];  
    int i = esq, j = meio;  
    for (int k = 0; k < tmp.length; k++) {  
        if (array[i] < array[j]) tmp[k] = array[i++];  
        else tmp[k] = array[j++];  
    }  
}
```

Questão 62 - Melhor e pior caso de trecho misto

Nível: Prova de reavaliação

Determine uma função para o número de multiplicações no melhor e no pior caso e descreva entradas que produzem cada caso. Considere $n > 0$.

```
for (int i = 0; i < n; i++) {  
    if (array[i] % 2 == 0) {  
        for (int j = 1; j <= n; j *= 2) x *= 2;  
    } else {  
        for (int j = 0; j < i; j++) x *= 3;  
    }  
}
```

Questão 63 - ABB: pesquisa e quantidade de comparações

Nível: Básico

Implemente public boolean pesquisar(int x) e faça o método também atualizar o atributo comparacoes com o número de nós comparados. Use auxiliar recursivo.

```
class No { int elemento; No esq, dir; }  
class ArvoreBinaria {  
    private No raiz;  
    private int comparacoes;  
    public boolean pesquisar(int x) {  
        // implementar  
    }  
}
```

Questão 64 - ABB: folhas e nós internos

Nível: Intermediário

Implemente public int contarFolhas() e public int contarInternos() em uma ABB. Cada método público deve chamar um auxiliar privado. Informe a complexidade.

Questão 65 - ABB: altura e limite logarítmico

Nível: Avançado

Implemente public boolean isMax(double valor), que retorna true se altura $\leq$ valor * $\log_2(\text{qtdNos})$. A árvore vazia deve retornar true. Não há métodos prontos de altura ou quantidade.

Questão 66 - Validar ABB

Nível: Avançado

Implemente `public boolean isABB()` sem copiar os elementos para vetor. A validação precisa detectar violações causadas por descendentes, não apenas por filhos imediatos. Assuma chaves sem repetição.

Questão 67 - Subconjunto de menores na ABB

Nível: Prova de reavaliação

Implemente `public ArvoreBinaria obterSubconjuntoMenores(int x)`, que cria e retorna uma nova ABB contendo todos os elementos menores ou iguais a `x`. As árvores não podem compartilhar nós. Lance exceção se `x` não existir na árvore original.

Questão 68 - Árvore de árvores

Nível: Avançado

A primeira árvore é ordenada por letra; cada nó possui uma segunda ABB de palavras. Implemente `public boolean pesquisar(String s)` e `public int contarComTamanho(int tam)`.

```
class No { char letra; No esq, dir; No2 raiz2; }  
class No2 { String palavra; No2 esq, dir; }  
class ArvoreArvore { private No raiz; }
```

Questão 69 - Árvore de listas - agenda

Nível: Avançado

Cada nó da ABB representa a primeira letra e contém uma lista de contatos. Implemente `public boolean pesquisar(int cpf)`, procurando em todas as listas, e `public boolean pesquisarNome(String nome)`, usando a primeira letra para escolher apenas um nó da árvore.

```
class No { char letra; No esq, dir; Celula primeiro, ultimo; }  
class Celula { Contato contato; Celula prox; }  
class Contato { String nome; int cpf; }
```

Questão 70 - AVL: nível e fator

Nível: Básico

Complete os métodos de `No` usando a convenção do material.

```
class No {  
    int elemento, nivel;  
    No esq, dir;  
    public void setNivel() { /* completar */ }
```

```
public static int getNivel(No no) { /* completar */ }  
public int getFatorBalanceamento() { /* completar */ }  
}
```

Questão 71 - Validar AVL

Nível: Avançado

Implemente public boolean isAVL() que verifica simultaneamente: propriedade de ABB, níveis armazenados corretos e $|\text{fator}| \leq 1$ em todos os nós. Faça em $O(n)$, evitando recalculá-los para cada nó.

Questão 72 - Balanceamento AVL

Nível: Prova de reavaliação

Implemente o método privado balancear(No no) compatível com o código do professor. As rotações simples já existem. O método deve detectar os quatro casos, atualizar níveis e lançar exceção para fator inválido.

Questão 73 - Remoção em AVL

Nível: Avançado

Complete a remoção recursiva de AVL usando o maior da esquerda e garantindo que cada retorno aplique balancear. Explique por que balancear somente a raiz ao final é insuficiente.

Questão 74 - Árvore 2-3-4: validade

Nível: Avançado

Implemente public boolean valida() que verifica: qtdChaves entre 1 e 3; chaves internas ordenadas; quantidade correta de filhos; intervalos de chaves respeitados; todas as folhas no mesmo nível.

```
class No234 {  
    int qtdChaves;  
    int[] chave = new int[3];  
    No234[] filho = new No234[4];  
}  
class Arvore234 { private No234 raiz; }
```

Questão 75 - Alvinegra: brancos dos lados da raiz

Nível: Prova de reavaliação

Considere $\text{cor} == \text{false}$ para branco e $\text{cor} == \text{true}$ para preto/colorido. Implemente public boolean mesmaQuantidadeBrancos() que compara a quantidade de nós brancos nas subárvores esquerda e direita da raiz.


```
class NoAN { boolean cor; int elemento; NoAN esq, dir; }
class Alvinegra { private NoAN raiz; }
```

Questão 76 - Alvinegra: propriedade global de brancos

Nível: Desafio

Implemente public boolean equilibradaBrancaGlobal(): para todo nó, a quantidade de nós brancos na subárvore esquerda deve ser igual à da direita. Sua solução deve ser $O(n)$, retornando boolean e contagem na mesma travessia.

Questão 77 - Alvinegra: invariantes

Nível: Avançado

Implemente public boolean verificaArvore() que confirma: raiz branca; nenhum nó preto/colorido possui filho preto/colorido; todos os caminhos até referências nulas têm a mesma quantidade de nós brancos. Faça uma única travessia.

Questão 78 - Hash com área de reserva

Nível: Intermediário

Implemente pesquisar e remover na hash com área de reserva. Ao remover um elemento da área principal, se houver na reserva um elemento que possui a mesma posição hash, mova o primeiro deles para a posição principal e compacte a reserva.

```
class Hash {
    int[] tabela;
    int m1, m2, reserva;
    final int NULO = -1;
    int h(int x) { return x % m1; }
}
```

Questão 79 - Hash com rehash

Nível: Intermediário

Implemente inserir, pesquisar e remover para uma tabela com duas tentativas: $h(x)=x\%m$ e $reh(x)=(x+1)\%m$. Use NULO e REMOVIDO distintos para não quebrar pesquisas após remoção.

Questão 80 - Hash com encadeamento

Nível: Intermediário

Implemente em hash indireta: inserirInicio, pesquisar, remover e int maiorLista(), que devolve o maior comprimento entre as listas. Analise complexidade média e pior caso.

Questão 81 - TRIE: palavra e prefixo

Nível: Intermediário

Com a representação do material, implemente `public boolean pesquisar(String s)` e `public boolean existePrefixo(String p)`. A primeira exige `folha=true`; a segunda retorna `true` se o caminho do prefixo existir e houver pelo menos uma palavra abaixo.

```
class No { char elemento; No[] prox = new No[255]; boolean folha; }  
class ArvoreTrie { private No raiz; }
```

Questão 82 - TRIE: prefixo e sufixo

Nível: Avançado

Implemente `public boolean existePrefixoSufixo(String prefixo, String sufixo)`. Navegue até o prefixo e explore somente sua subárvore. Não mantenha uma lista global pronta de palavras. Informe a complexidade em termos dos caracteres visitados.

Questão 83 - PATRICIA: pesquisar e contar A

Nível: Avançado

A PATRICIA guarda segmentos por índices *i,j,k* em um vetor de palavras. Implemente `pesquisar(String s)` e `contarAs()`, contando letras A nos rótulos armazenados, sem contar uma mesma posição mais de uma vez.

```
class No { int i, j, k; boolean folha; No[] prox; }  
class Patricia { String[] array; No raiz; }
```

Questão 84 - Estrutura doidona

Nível: Desafio

Implemente `public boolean pesquisar(int elemento)` sem usar classes prontas. A estrutura funciona assim: primeiro tenta T1; em colisão, `hashT2` escolhe entre T3, lista e ABB. Em T3, tenta `hash`, `rehash` e, persistindo a colisão, outra ABB. Use apenas os atributos fornecidos. Depois indique a complexidade por camada.

```
class Celula { int elemento; Celula prox; }  
class No { int elemento; No esq, dir; }  
class Doidona {  
    int[] t1, t3;  
    Celula primeiroLista, ultimoLista;  
    No raizArvoreT2, raizArvoreT3;  
    int hashT1(int x) { /* pronta */ return 0; }  
    int hashT2(int x) { /* pronta */ return 0; }  
    int hashT3(int x) { /* pronta */ return 0; }
```

```
int rehashT3(int x) { /* pronta */ return 0; }  
}
```

Simulados finais

Simulado 1 - Nível semelhante à primeira reavaliação

1. Dado um trecho com dois laços dependentes, escreva o somatório do número de multiplicações, obtenha a fórmula fechada e prove por indução.
2. Insira 11, 4, 18, 2, 7, 15, 20, 6, 9, 13, 17 em: 2-3-4 com fragmentação na descida, AVL e alvinegra. Mostre o passo a passo.
3. Prove ou refute oito afirmações sobre lista, árvores balanceadas, Merge Sort, Radix Sort e hash.
4. Modifique Insertion Sort para deixar múltiplos de 3 antes dos demais, ordenando cada grupo de forma crescente.
5. Analise melhor e pior caso de um código que alterna laços lineares, logarítmicos e condicionais.
6. Implemente boolean pesquisar(String palavra) em uma estrutura doidona formada por árvore de caracteres, hash por tamanho e lista/ABB para colisões.

Simulado 2 - Conteúdos mais prováveis

1. Implemente Bubble Sort com interrupção antecipada e última troca; derive o somatório de comparações no pior caso.
2. Dada uma ABB, implemente uma nova ABB apenas com valores do intervalo [a,b], sem compartilhar nós.
3. Em uma matriz encadeada, retorne uma lista dupla com os elementos da diagonal secundária.
4. Implemente inserção ordenada e remoção por valor em lista simples.
5. Desenhe uma AVL após uma sequência que gere rotação simples e dupla; escreva o código de balancear.
6. Prove ou refute afirmações envolvendo pilha, fila circular, lista dupla, ABB e tabela hash.

Simulado 3 - Preparação avançada

1. Implemente uma função boolean que valide simultaneamente ABB, AVL e níveis armazenados.
2. Desenhe uma 2-3-4 e sua representação alvinegra após 12 inserções; destaque fragmentações e recolorações.
3. Implemente pesquisa por prefixo e sufixo em uma TRIE cujos filhos são mantidos em uma ABB.
4. Implemente remoção em hash com reserva e realocação de um elemento compatível da reserva para a área principal.

5. Em uma árvore de árvores, conte palavras com tamanho par somente nos nós da primeira árvore cuja letra esteja em um intervalo.
6. Analise e implemente pesquisa em uma estrutura com quatro camadas: AVL -> hash -> lista -> TRIE.